

Torch2Gemmini:
End-to-End Deployment of PyTorch Models on FPGA-Based
Gemmini Accelerator

Noel Manuel
EP22B035

UGRC
IIT Madras

Contents

1	Introduction	2
1.1	Problem Statement and Objectives	2
1.2	Contributions	2
2	Background and Existing Systems	2
2.1	Chipyard Ecosystem	3
2.1.1	Rocket Processor	3
2.1.2	Gemmini Accelerator	3
2.1.3	FPGA Deployment	3
2.2	Compilation Framework	4
2.2.1	LLVM	4
2.2.2	MLIR	4
2.2.3	Torch-MLIR	4
2.2.4	Buddy-MLIR	5
3	Compilation Flow and System Architecture	5
3.1	Model training and Quantization	6
3.2	MLIR Generation	7
3.3	Compiler Lowering and Execution Pipeline	7
4	Experimental Evaluation	7
4.1	Experimental Setup	7
4.2	Matrix Multiplication Results	8
4.3	MLP Inference on MNIST	8
4.4	LeNet Inference Results	8
4.5	Performance Analysis	9
5	Conclusion	9

1. Introduction

Deep learning workloads such as matrix multiplication and convolution require high computational throughput and efficient data movement. While CPUs provide flexibility, they are not optimized for the massive parallelism required by modern neural network inference workloads. As a result, specialized accelerators such as systolic-array-based architectures have become increasingly important for improving performance and energy efficiency.

Gemmini[1] is an open-source deep learning accelerator generator integrated within the RISC-V Chipyard ecosystem[2]. It provides configurable hardware support for low-precision neural network operations, particularly int8 matrix multiplication with int32 accumulation. At the software level, frameworks such as LLVM[3], MLIR[4], Torch-MLIR [5], and Buddy-MLIR[6] provide compiler infrastructures for lowering high-level machine learning programs toward hardware-specific implementations.

1.1. Problem Statement and Objectives

Machine learning models are commonly developed using high-level frameworks such as PyTorch. Deploying PyTorch models on FPGA-based accelerators remains difficult due to challenges in compiler lowering, datatype handling, runtime integration, and hardware-specific code generation. Existing Gemmini applications are often manually implemented in C/C++, limiting generality and scalability. This project focuses on building an end-to-end framework that integrates Torch-MLIR, Buddy-MLIR, LLVM, Chipyard to execute quantized PyTorch models on FPGA hardware.

1.2. Contributions

The major contributions of this project are:

- Built a complete PyTorch-to-Gemmini compilation and deployment pipeline.
- Added support for the `torch._int_mm` operator in Torch-MLIR.
- Modified the `convert-linalg-to-gemmini` pass in Buddy-MLIR to recover original int8 tensors from sign-extended int32 buffers.
- Extended support for VC707 FPGA board in Chipyard by adding UART-TSI flow for loading programs and enabled execution of quantized neural network workloads on VC707.
- Demonstrated significant acceleration using Gemmini, including approximately $638\times$ speedup for MLP inference and approximately $44\times$ speedup for LeNet inference.

2. Background and Existing Systems

This section describes the hardware and software ecosystem used in this project. The first subsection explains the Chipyard-based hardware platform consisting of the Rocket processor, Gemmini accelerator, and FPGA deployment flow. The second subsection explains the compiler infrastructure used to lower PyTorch models into Gemmini-executable RISC-V binaries.

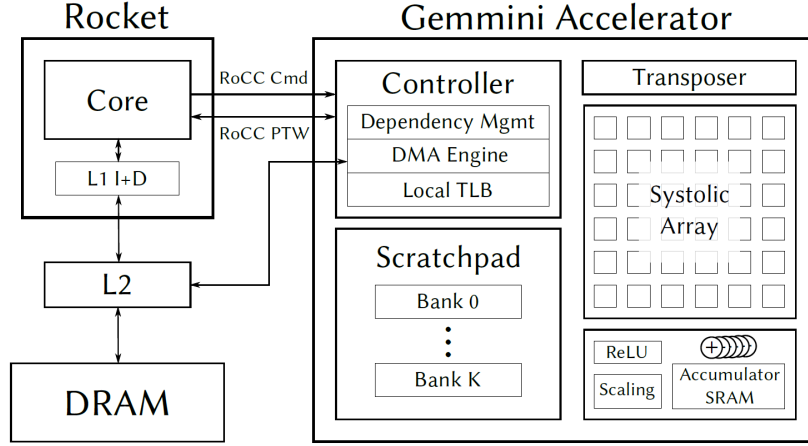


Figure 1: Rocket + Gemmini

2.1. Chipyard Ecosystem

Chipyard[2] is an open-source framework for designing and prototyping RISC-V-based Systems-on-Chip (SoCs). It provides an integrated environment for processor generation, accelerator integration, simulation, and FPGA deployment. In this project, Chipyard was used to build a system containing the Rocket processor and the Gemmini deep learning accelerator.

2.1.1. Rocket Processor

Rocket is a RISC-V in-order processor core generated using the Chisel hardware construction language. It forms the main CPU of the system and executes the software responsible for controlling Gemmini operations.

Rocket supports the RoCC (Rocket Custom Coprocessor) interface, which allows custom accelerators to be attached directly to the processor pipeline. Gemmini communicates with the Rocket core through this interface. The CPU is responsible for loading inputs, configuring accelerator operations, and handling execution flow, while computationally intensive matrix operations are offloaded to Gemmini.

2.1.2. Gemmini Accelerator

Gemmini[1] is a systolic-array-based deep learning accelerator integrated within Chipyard. It is designed for accelerating matrix multiplication and convolution workloads commonly used in neural network inference. Gemmini internally uses systolic arrays, where multiply-accumulate operations are distributed across a grid of processing elements. This architecture enables high throughput and efficient reuse of data during matrix computations.

2.1.3. FPGA Deployment

The generated Rocket + Gemmini hardware design was deployed on FPGA platforms for real hardware execution and testing. Initially, the Nexys Video board was explored because Chipyard already provided UART-TSI support for loading programs onto the FPGA. However, the Nexys Video platform has limited FPGA resources compared to VC707. The VC707 board was later selected as the primary deployment platform because it supports larger Gemmini configurations

and more complex workloads. However, the default Chipyard deployment flow for VC707 uses an SD-card-based mechanism for loading executable binaries. In this approach:

- The RISC-V ELF executable is copied into an SD card.
- The FPGA boots Linux or a bootloader environment.
- The executable is loaded from the SD card into memory and executed.

During development, this pathway repeatedly failed despite multiple attempts using different approaches forcing us to use UART-TSI pathway. UART-TSI (Tethered Serial Interface) provides an alternative mechanism for loading programs directly from the host computer to the FPGA over UART communication. In this flow:

- The host computer sends the ELF executable through UART.
- The TSI protocol transfers the binary into FPGA memory.
- The program begins execution directly without requiring an SD card.

Chipyard already supported UART-TSI for Nexys Video, but this support was not available for VC707 by default. Therefore, configuration files for VC707 were modified to enable UART-TSI functionality. After enabling this flow, ELF binaries could be loaded and executed successfully on the VC707 FPGA. This significantly simplified debugging and accelerated the development workflow.

2.2. Compilation Framework

This project uses an MLIR-based compilation framework to convert PyTorch models into Gemmini-executable RISC-V binaries. The compilation stack consists Torch-MLIR and Buddy-MLIR.

2.2.1. LLVM

LLVM (Low Level Virtual Machine)[3] is a modular compiler infrastructure used for program optimization and machine-code generation. It uses LLVM IR (Intermediate Representation), a low-level platform-independent representation of programs. It provides backend support for generating machine code for multiple architectures, including RISC-V.

2.2.2. MLIR

MLIR (Multi-Level Intermediate Representation)[4] extends LLVM by introducing multiple abstraction levels within the compilation process. Instead of directly lowering high-level programs into low-level LLVM IR, MLIR allows transformations through several intermediate dialects. Each dialect represents operations at a different level of abstraction. This multi-level lowering is especially useful for machine learning workloads because tensor operations can be preserved and optimized before conversion into low-level instructions.

2.2.3. Torch-MLIR

Torch-MLIR[5] provides a frontend for converting PyTorch programs into MLIR representations. In this project, PyTorch models written in Python were first converted into MLIR using

Torch-MLIR. The generated MLIR primarily used the linalg dialect for representing tensor operations such as matrix multiplication and convolution.

The linalg dialect represents structured linear algebra computations in MLIR. Operations such as `linalg.matmul` are expressed in a form that preserves tensor semantics and iteration structure. For example, matrix multiplication generated from PyTorch can be represented as:

```
%2 = linalg.matmul ins(%arg0, %arg1 : tensor<8x8xi8>, tensor<8x8xi8>)
outs(%1 : tensor<8x8xi32>) -> tensor<8x8xi32>
```

The linalg dialect is important because it provides a high-level representation suitable for accelerator-aware transformations. However, Torch-MLIR initially widened int8 tensors into int32 before matrix multiplication, which prevented proper Gemmini acceleration and caused fallback to CPU execution. Support for the `torch.int_mm` operator was later added so that quantized int8 matrix multiplication could be correctly represented in the lowering flow.

2.2.4. Buddy-MLIR

Buddy-MLIR[6] extends MLIR with accelerator-oriented lowering flows and additional dialects for hardware acceleration. In this project, Buddy-MLIR was responsible for lowering linalg operations into Gemmini-specific operations and finally into LLVM-compatible representations.

Buddy-MLIR introduces the Gemmini dialect containing operations specific to the Gemmini accelerator. These operations abstract Gemmini hardware instructions at a higher level before final lowering into LLVM dialect and machine code. One important operation in this dialect is `gemmini.tile_matmul` which represents tiled matrix multiplication mapped onto the Gemmini accelerator. Two major passes from buddy were used in this project -

- `convert-linalg-to-gemmini` - This pass converts high-level linalg operations into Gemmini dialect operations. `linalg.matmul` is transformed into `gemmini.tile_matmul`.
- `lower-gemmini` - After conversion into Gemmini dialect operations, this pass lowers Gemmini-specific operations into lower-level MLIR representations compatible with LLVM.

The resulting MLIR is then converted into LLVM dialect, translated into LLVM IR, and finally compiled into RISC-V object files using LLVM backend tools. The final object files are linked with runtime wrapper code to generate executable ELF binaries capable of running on Rocket + Gemmini FPGA systems.

3. Compilation Flow and System Architecture

This section describes the complete workflow developed in this project for executing quantized PyTorch models on Gemmini-enabled RISC-V systems. The framework integrates PyTorch, Torch-MLIR, Buddy-MLIR, LLVM, and the Chipyard ecosystem to generate FPGA-executable binaries from neural network models.

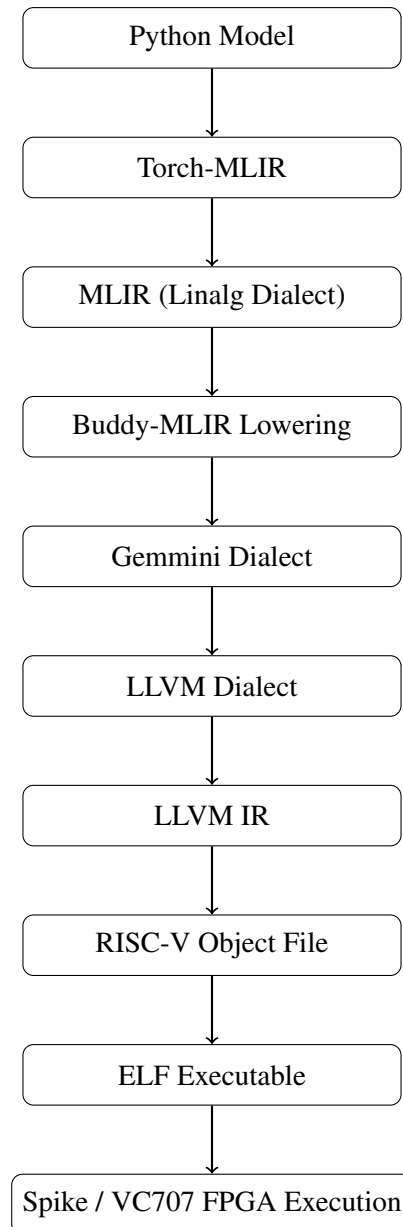


Figure 2: Compilation and execution flow from PyTorch model to Gemmini-enabled RISC-V execution

3.1. Model training and Quantization

Gemmini primarily targets low-precision inference workloads using int8 multiplication and int32 accumulation. Therefore, neural network models must be quantized before deployment. In this project, post-training quantization was used. The model was first trained in floating-point precision using PyTorch and later quantized into int8 representations suitable for Gemmini execution.

For the LeNet workflow:

- `train.py` trains the LeNet model on the MNIST dataset.
- The trained floating-point parameters are quantized into int8 values.
- Quantized weights and biases are exported into: `lenet_quantized.json`

Quantization is necessary because the default Gemmini configuration performs int8 multiplication with int32 accumulation. Using int8 values significantly reduces memory usage, bandwidth requirements, and computation cost while maintaining acceptable inference accuracy. A major challenge encountered during development was that Torch-MLIR widened int8 tensors into int32 before matrix multiplication. This prevented Gemmini execution and caused fallback to CPU execution. The lowering flow was therefore modified to recover original int8 tensors during lowering.

3.2. MLIR Generation

MLIR generation forms the frontend stage of the compilation pipeline. The file `:generate_mlir.py` is responsible for generating the MLIR representation of the LeNet model. This script loads quantized weights from `lenet_quantized.json`, constructs the neural network and expresses all the matrix multiplications in terms of `torch._int_mm` to generate the MLIR code. The generated MLIR primarily uses the `linalg` dialect. Operations such as matrix multiplication and convolution are represented using `linalg.matmul`.

3.3. Compiler Lowering and Execution Pipeline

The compiler lowering pipeline converts MLIR tensor operations into executable RISC-V binaries capable of running on Gemmini-enabled hardware. `linalg.matmul` is converted to `gemmini.tile_matmul` using the pass `convert-linalg-to-gemmini`.

Further lowering to lower level MLIR operations is done by `lower-gemmini`. Later the mlir code is converted to LLVM IR using `buddy-translate` tool(which is a patched version of `mlir-translate`).Then `buddy-llc` tool converts the LLVM code to object file. This object file is later linked with wrapper code to generate executable ELF binaries.`wrapper.c` contains the code responsible for passing inputs to the generated functions, handling memory allocation,printing outputs and measuring cycle counts.

The input image for inference is generated using `image.py` which generates the `input.bin` which is later converted to a c header file `input_data.h`.It is imported by the c wrapper during the linking of wrapper with object code. The final ELF binary can then execute on Spike or FPGA hardware.

4. Experimental Evaluation

This section presents the experimental evaluation of the developed Torch2Gemmini framework. The generated binaries were executed both with and without Gemmini acceleration in order to compare performance improvements achieved using the systolic-array accelerator.The github repository is open sourced [7].

4.1. Experimental Setup

The hardware platform used in this project consisted of a Rocket processor integrated with the Gemmini accelerator within the Chipyard ecosystem.

The generated Gemmini configuration used:

- A 16×16 systolic array
- int8 multiplication and int32 accumulation

The Rocket + Gemmini bitstream was generated targeting a clock frequency of 25 MHz. The generated hardware design was synthesized and deployed onto the VC707 FPGA using Xilinx Vivado. Detailed steps for FPGA programming and deployment is in the GitHub repository.

4.2. Matrix Multiplication Results

The first experiment evaluated matrix multiplication performance using two 8×8 matrices.

Table 1: Matrix Multiplication Performance

Configuration	Cycle Count
CPU Only	38593
Gemmini + CPU	4678

$$Speedup = \frac{CPUCycles}{GemminiCycles} = \frac{38593}{4678} \approx 8.25 \times$$

The results demonstrate that even for relatively small matrix sizes, Gemmini provides noticeable acceleration compared to CPU-only execution. Matmul for higher dimensional matrices would give considerably more speed up than 8×8 matrices.

4.3. MLP Inference on MNIST

A three-layer multilayer perceptron (MLP) model with architecture:

$$784 \rightarrow 128 \rightarrow 10$$

was trained on the MNIST dataset and quantized into int8 format.

Inference was performed on a single MNIST image input.

Table 2: MLP Inference Performance

Configuration	Cycle Count
CPU Only	27172259
Gemmini + CPU	42600

$$Speedup = \frac{27172259}{42600} \approx 637.8 \times$$

The results demonstrate the effectiveness of Gemmini acceleration for quantized neural network inference workloads.

4.4. LeNet Inference Results

A quantized LeNet convolutional neural network trained on the MNIST dataset was also evaluated using the developed framework.

Inference was performed for a single image input.

Table 3: LeNet Inference Performance

Configuration	Cycle Count
CPU Only	73482069
Gemmini + CPU	1674705

$$Speedup = \frac{73482069}{1674705} \approx 43.87 \times$$

The reduced speedup compared to the MLP workload is expected because LeNet includes additional convolution, activation, and memory operations that introduce overheads beyond matrix multiplication alone.

4.5. Performance Analysis

The experimental results demonstrate that Gemmini significantly accelerates neural network inference workloads compared to CPU-only execution. The primary reason for the observed acceleration is Parallel execution using systolic arrays. Gemmini performs matrix multiplications using a tiled systolic-array architecture where multiple multiply-accumulate operations occur simultaneously across processing elements. This allows dense linear algebra workloads to execute far more efficiently than sequential CPU execution. The accelerator also minimizes expensive DRAM accesses by reusing data inside scratchpad memories and accumulation buffers. This becomes particularly beneficial for convolution and matrix multiplication workloads where activations and weights are repeatedly reused.

5. Conclusion

This project presented Torch2Gemmini, an end-to-end framework for deploying quantized PyTorch models onto FPGA-based Gemmini accelerator. The work integrated Torch-MLIR, Buddy-MLIR, LLVM, and the Chipyard ecosystem to create a complete compilation and deployment pipeline capable of generating Gemmini-aware RISC-V executables from high-level Python models.

The developed framework was successfully used to execute matrix multiplication, MLP inference, and LeNet inference workloads on a VC707 FPGA configured with Rocket + Gemmini. Experimental results demonstrated substantial acceleration compared to CPU-only execution, including approximately $638\times$ speedup for MLP inference and approximately $43.87\times$ speedup for LeNet inference.

The project demonstrates the effectiveness of combining MLIR-based compiler infrastructures with open-source hardware accelerators for machine learning deployment. It also highlights the importance of hardware-software co-design in enabling efficient execution of quantized neural network workloads on specialized accelerators. Future improvements can focus on expanding Torch-MLIR operator coverage, automating runtime wrapper generation, supporting larger neural network models, and improving generalization of the compilation flow for arbitrary PyTorch programs.

References

- [1] H. Genc, A. Behroozi, Y. S. Shao *et al.*, “Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration,” in *Proceedings of the 58th Annual Design Automation Conference (DAC)*, 2021.
- [2] Chipyard Project, “Chipyard documentation,” <https://chipyard.readthedocs.io>, 2026, accessed: 2026-05-21.
- [3] LLVM Project, “Llvm documentation,” <https://llvm.org/docs/>, 2026, accessed: 2026-05-21.
- [4] C. Lattner, J. Pienaar, M. Amini *et al.*, “Mlir: A compiler infrastructure for the end of moore’s law,” *arXiv preprint arXiv:2002.11054*, 2020. [Online]. Available: <https://arxiv.org/abs/2002.11054>
- [5] Torch-MLIR Project, “Torch-mlir documentation,” <https://github.com/llvm/torch-mlir>, 2026, accessed: 2026-05-21.
- [6] Buddy-MLIR Project, “Buddy-mlir repository,” <https://github.com/buddy-compiler/buddy-mlir>, 2026, accessed: 2026-05-21.
- [7] N. Manuel, “Torch2gemmini,” <https://github.com/NMJ0/Torch2Gemmini.git>, 2026, gitHub repository.